

by small differences in word derivation; we want to consider a sentence about jumping to be comparable to a sentence about a jumper, even though they are technically different words. Stemming removes the ends of derived words to convert them into standardized word stems. A function for stemming is available in Python's `nltk` module, and we can use this function with a list comprehension as follows:

```
stemmer = nltk.stem.porter.PorterStemmer()
def stem_tokens(tokens):
    return [stemmer.stem(item) for item in tokens]
```

In this snippet, we've created a function called `stem_tokens()`. It takes a list of tokens and calls `nltk`'s `stemmer.stem()` function to turn them into stems:

```
print(stem_tokens(nltk.word_tokenize(query.lower().translate(remove_punctuation_map))))
```

The output is `['i', 'want', 'to', 'learn', 'about', 'geometri', 'algorithm']`. Our stemmer has converted *algorithms* to *algorithm* and *geometry* to *geometri*. It has replaced a word with what it regards as its stem: a singular word or word portion that will make text comparisons easier. Finally, we put our normalization steps together in one function, `normalize()`:

```
def normalize(text):
    return
    stem_tokens(nltk.word_tokenize(text.lower().translate(remove_punctuation_map)))
```

Text Vectorization

Now you're ready to learn how to convert texts to numeric vectors. It's easier to make quantitative comparisons between numbers and vectors than between words, and we'll need to make quantitative comparisons to make our chatbot work.

We'll use a simple method called *TFIDF*, or *term frequency-inverse document frequency*, which converts documents into numeric vectors. Each document vector has one element for each term in a corpus. Each element is the product of the term frequency for a given term (a raw count of the number of times the term occurs in a particular document) and the inverse